

Class 01 — Introduction to Front-End

TAP Academy — HTML & CSS

Class 01: Introduction to Front-End

1. What Front-End Means

- Front-end is the part of an application users directly see and interact with.
- Buttons, text, images, layouts, colors, spacing, media players, navigation and other browser-visible UI are built through front-end technologies.
- Front-end development is practical: projects matter, not just memorized definitions.
- Core front-end foundation: **HTML + CSS + JavaScript**. React and other frameworks build on these fundamentals.

2. Why Learn HTML & CSS in the AI Era?

- AI tools can generate complete front-end pages, but developers still need to:
 - understand the fundamentals,
 - verify generated code,
 - identify and fix problems,
 - describe the required UI accurately.
- AI reduces typing effort; it does not remove the need for understanding.
- HTML and CSS alone are not enough for a modern front-end developer. React/other frameworks and JavaScript are also important.
- Strong HTML/CSS fundamentals make learning React easier.

3. Browser, Client, Server — Foundation

Client

- The user's device/browser: laptop, desktop or mobile.
- The browser displays the content returned by the server.

Server

- A computer/system with much greater processing power, memory and storage, designed to run applications and handle many users.
- Application data and processing are generally handled on servers rather than stored entirely on the user's device.

Request → Server → Response

1. The client sends a request to a server.
2. The server processes the request.
3. The server sends a response.
4. The browser displays the returned content.

This request/response cycle happens repeatedly while using web applications.

4. What the Browser Receives

- A web page can contain text, images, audio, video and other content.
- HTML provides the structure/content that the browser interprets and displays.
- Developer tools/Inspect can expose and temporarily modify the HTML/CSS loaded in the local browser.
- Such local modifications do **not** change the actual server-side data. Refreshing the page loads the server response again.

5. Front-End vs Back-End

- **Front-end:** what the user sees and interacts with in the browser.
- **Back-end:** server-side processing, application logic and data handling.
- A browser can expose front-end HTML/CSS/JavaScript through developer tools, but server-side/back-end logic is not exposed in the same way.

6. Website vs Web Application

Web page

- A single page/document available through the web.

Website

- A collection of web pages.
- In the class example, Wikipedia was used as a simple example of mostly fixed/static content.

Web application

- A web-based application whose content/behavior can change based on the user or actions.
- Examples discussed: Flipkart and Amazon.
- Web applications commonly involve back-end processing and databases for features such as accounts, payments and personalized content.

Key distinction

- Static/fixed content → website-style content.
- Dynamic/user-specific content → web application.
- Both websites and web applications are made from web pages.

7. HTML — Core Concept

HTML = HyperText Markup Language

- HTML is **not a programming language**.
- HTML is a **markup language**.
- It is used to structure and place content in a browser.
- HTML can define where and what type of content should appear: headings, text, images, audio, video and other elements.
- Do not treat HTML as Java/Python: it does not provide programming constructs such as loops, if/else logic, classes or data structures.

8. Markup Language

A markup language uses markup/tags to tell the browser how content should be structured or represented.

For HTML:

- Tags communicate the intended structure/content to the browser.
- Content goes inside the relevant HTML structure.
- HTML focuses on **structure and content**, not application logic.

9. HTML Tags

Basic idea:

```
<tag>content</tag>
```

- **Opening tag:** `<tag>`
- **Content:** the text/media/etc. placed inside
- **Closing tag:** `</tag>`
- The closing tag normally contains a forward slash `/`.

Example:

```
<h1>Front End</h1>
```

- `h1` represents a heading.
- HTML elements are built using tags and content.

10. HTML + CSS

Think of:

- **HTML** → **structure/content**
- **CSS** → **presentation/style**

CSS controls visual aspects such as:

- colors,
- fonts,
- spacing,
- sizing,
- borders,
- rounded corners,
- background,
- layout/presentation.

Examples mentioned in class:

- HTML creates a heading or image.
- CSS controls how the heading looks and how much spacing surrounds elements.

11. Important HTML/CSS Terms Introduced

You should be able to recognize and later explain:

- `div`
- `class`
- `id`
- `h1`
- margin
- padding
- box model
- `box-sizing: border-box`
- CSS selectors
- attribute selectors
- pseudo-classes

The class emphasized that knowing HTML/CSS means understanding what these pieces of code actually do, not merely recognizing their names.

12. HTML Evolution — High-Level

The class gave a brief historical progression:

- HTML 1.0
- HTML 2.0
- HTML 3.2
- HTML 4, where CSS became part of the modern web development workflow
- XHTML
- HTML5, introduced as the modern major version

HTML5 expanded modern web capabilities, including native support for media such as audio and video.

13. Development Environment

- HTML can be written in different code editors.
- **Visual Studio Code (VS Code)** was recommended for the course.
- The instructor said the initial classes will focus on writing HTML from scratch before using AI-assisted editors.
- The class setup uses an HTML project/folder and an HTML file such as `index.html`.

14. What the Course Will Build

The instructor demonstrated that HTML/CSS can be used to build pages containing:

- text,
- images,
- videos,
- audio,
- buttons,
- borders,
- rounded corners,

- layouts and spacing.

The focus is on building these features from scratch and understanding how they work.

15. Key Takeaways

1. Front-end is the browser-facing part of an application.
2. **HTML = structure/content.**
3. **CSS = presentation/style.**
4. HTML is a markup language, **not** a programming language.
5. Understand client, server, request and response before memorizing HTML tags.
6. Website content can be static/fixed; web applications commonly provide dynamic/user-specific content.
7. Developer Tools can inspect and temporarily modify the HTML/CSS loaded in your browser.
8. AI can generate code, but you still need enough fundamentals to understand, verify and direct that code.
9. HTML/CSS are foundational for learning React and modern front-end development.
10. Do not memorize definitions blindly; understand what the technology actually does.

16. Before the Next Class

- Install **VS Code**.
- Create a dedicated HTML practice folder.
- Create an `index.html` file.
- Research each part of the basic HTML document structure before the next class.
- Be able to explain: **client, server, request, response, HTML, markup language, tag, HTML vs CSS.**

Day 02 — Multimedia Blog Webpage

TAP Academy — HTML & CSS

Day 02: Multimedia Blog Webpage

1. Core concepts

- **Client** = the user's device/browser.
- **Server** = a computer running the application/software and handling requests.
- Computers communicate using **IP addresses**.
- Windows Command Prompt: `ipconfig` can show IPv4, IPv6 and default gateway information.
- Because server IP addresses are difficult to remember, **domain names** provide human-readable names.
- HTML = **HyperText Markup Language** and is a markup language, not a programming language.
- HTML is **rendered by the browser**. The class emphasized saying *render*, not *execute*.
- The browser can also display PDFs/images and play audio/video.

2. VS Code project setup

- Create a project folder and open the folder in VS Code.
- Use `index.html` as the common website entry point.
- HTML output is viewed in a browser, not the Java/Python-style Run button.
- In VS Code, `!` + Enter generates the HTML boilerplate.
- Another shortcut demonstrated: `html:5`.
- **Boilerplate** = predefined/basic starting structure.
- HTML's basic structure contains `<!DOCTYPE html>`, `html`, `head`, and `body`.
- `html` is the **root element**.
- `head` contains information such as the page title; `body` contains visible page content.
- To change the browser tab title, use `<title>...</title>` inside `<head>`.

3. Live Server

- Install the **Live Server** VS Code extension.
- Right-click the HTML file → **Open with Live Server**, or click **Go Live**.
- Save changes with **Ctrl+S**.
- Live Server updates the browser preview without manually refreshing after every change.
- The browser used by Live Server depends on the system's default browser.

4. HTML content tags

- `<h1>` → largest heading; `<h2>`, `<h3>`, etc. provide smaller heading levels through `<h6>`.
- `<p>` → paragraph.

- `<button>` → button.
- Visible content belongs inside `<body>`.
- HTML is used for structure/content; CSS is used for presentation, spacing and visual styling.

```
<!DOCTYPE html>
<html>
<head>
  <title>Day 2</title>
</head>
<body>
  Welcome to HTML
</body>
</html>
```

```
<style>
body {
  background-color: black;
  color: aqua;
}
</style>
```

```
<h1>Music I love is the biggest Indian blockbuster</h1>

<p>Some paragraph content...</p>

<button>Play the songs</button>

<h2>My favorite songs of all time</h2>

<h3>Song 1</h3>
<h3>Song 2</h3>
<h3>Song 3</h3>
```

```
button {
  background: green;
  border-radius: 15px;
  padding: 10px 5px;
}
```

5. CSS examples demonstrated

The instructor briefly used CSS directly inside `<style>` to demonstrate:

- `background-color`
- `color` for text color

- `background`
- `border`
- `border-radius`
- `padding`
- `width`
- CSS can change appearance without changing the HTML content.
- Spacing/layout control was intentionally deferred to the CSS lessons.

6. VS Code shortcut

- **Shift + Alt + Down Arrow** → duplicate the current line downward.
- The instructor used it to quickly create Song 2 and Song 3 from Song 1.

7. HTML attributes

- An **attribute** gives extra information to an HTML element.
- Examples demonstrated: `src`, `controls`, `height`, `width`, `lang`, `style`, `poster`.
- The instructor noted that in React, similar information is commonly called **props**.
- Attribute order does not matter; attributes need separation by spaces.

8. Audio

- `<audio>` places audio content.
- `src` specifies the audio source.
- `controls` gives the browser's built-in play/pause and playback controls.
- Without `controls`, the audio may exist but visible playback controls are not provided.
- MP3 and OGG were mentioned as audio formats.
- The browser creates the actual playback UI; you do not manually code every play/pause control.

9. Local assets and relative paths

- Keep downloaded audio, image and video files inside the project folder.
- `./` means the current folder.
- Example: `./asset/...` means enter the `asset` folder from the current location.
- The class demonstrated moving assets into the project using drag-and-drop.
- Correct folder placement is necessary for the relative path to work.

```
<h3>Song 1</h3>
<audio src="./asset/song1.mp3" controls></audio>

<h3>Song 2</h3>
<audio src="./asset/song2.mp3" controls></audio>
```

10. Images

- `<img>` places an image.
- `src` is the image source.
- `height` and `width` control dimensions.
- The instructor described `<img>` as a **self-closing/void element**: it does not have a separate `</img>` closing tag.
- Images can come from a downloaded local file or a direct image URL.
- For a remote image, the instructor demonstrated **Copy image address** and pasting that URL into `src`.
- A remote image depends on the original source remaining available.
- **Landscape** = wider/horizontal; **portrait** = taller/vertical.
- Image dimensions in the examples were measured in **pixels**.
- Choose appropriate height/width values rather than distorting the image.

11. Pixels and display resolution

- Screen/image dimensions are commonly represented in pixels.
- PPI = pixels per inch.
- The class connected higher pixel density/resolution with greater visual detail.
- Common examples discussed: 480p, 720p, 1080p, 4K and 8K.
- 16:9 was described as a common screen/video aspect ratio.
- These display concepts were used to explain why HTML media dimensions use pixel values.

12. Video

- `<video>` places video content.
- `src` specifies the video file.
- `controls` enables browser playback controls.
- `height` / `width` can control dimensions.
- Browser controls can provide play/pause, volume, fullscreen, download, playback speed and picture-in-picture depending on the browser.
- The browser provides these controls; you do not manually code each control.
- **Poster/thumbnail**: the `poster` attribute places an image shown before the video starts.
- The `poster` path can use the same relative-path approach as other local assets.

13. Relative vs absolute path

- A path such as `./asset/thumbnail11...` was demonstrated as a **relative path** because it starts from the current project location.
- An absolute path gives the full location from the drive/root.
- For project work, relative paths are the practical approach shown in the class.

```

```

```

```

```
<video src="./asset/trailer1.mp4" controls height="350"></video>
```

```
<video  
  src="./asset/trailer1.mp4"  
  controls  
  height="350"  
  poster="./asset/thumbnail1.jpg">  
</video>
```

14. Important multimedia pattern

The main pattern from the class is:

HTML tag + attributes + source/path

Examples:

- Audio → `<audio src="..." controls>`
- Image → ``
- Video → `<video src="..." controls ...>`
- Video thumbnail → add `poster="..."`

The instructor repeatedly emphasized that the browser supplies much of the built-in media functionality.

15. JavaScript glimpse — changing HTML

This was only a preview of JavaScript, not a full JavaScript lesson.

The instructor:

1. Gave the button an `id`, shown as `btn`.
2. Used `document.getElementById("btn")` to select it.
3. Stored the returned element in a `let` variable.
4. Used `innerText` to change the button text.

Important concepts introduced:

- `id` identifies an HTML element.
- `document.getElementById(...)` selects an element by its ID.
- `let` declares a variable.
- `innerText` changes the text inside the selected element.
- JavaScript can change HTML content without manually editing the HTML text.
- The instructor mentioned that the next step would be making the change happen when a function is called/clicked.

```
let btn = document.getElementById("btn");  
btn.innerText = "played song";
```

16. HTML → CSS → JavaScript → React

The instructor's learning order:

1. **HTML** — structure/content
2. **CSS** — styling/layout/responsiveness
3. **JavaScript** — behavior/interaction
4. **React** — framework/library-level front-end development

The point of learning HTML/CSS is not that developers must manually type basic HTML forever. The instructor emphasized learning the fundamentals so that you can understand, debug and give precise instructions to AI tools.

17. Forms — next important HTML area

The class identified forms as a major upcoming topic:

- Login forms
- Registration forms
- Contact forms
- Input fields
- Validation
- Password requirements/warnings

18. AI and fundamentals

- The instructor demonstrated that AI can generate/modify front-end code.
- However, you need the correct technical vocabulary to describe what you want.
- Examples: asking for a wider audio player requires knowing the term **width**; asking for a video thumbnail requires knowing **poster**.
- The instructor's practical message: first understand the fundamentals and build a few projects from scratch; then use AI heavily to accelerate development.

19. Class project / practice task

Build a multimedia blog page similar to the instructor's project.

Minimum task mentioned:

- At least **5 audio items**.
- Around **10 movie/video items** of your choice/language.
- Use headings and paragraphs.
- Use images from URLs and/or downloaded images.
- Use downloaded/local video assets.
- Organize assets inside the project.
- Do the project from scratch rather than blindly copying.
- YouTube links will not work as direct video files in the demonstrated approach; the instructor said a different YouTube embedding method would be covered later.
- Suggested sources mentioned for assets: Freepik, Pexels, Unsplash and Pinterest.
- Wikipedia was mentioned as a source for paragraph/content text.

20. Deployment — next class

- The project currently exists only on the local computer.
- To make it accessible through a link, it must be uploaded to a server.
- This process is called **deployment**.
- The next class will introduce deploying HTML pages and discuss **Git/GitHub**.

Key takeaways

1. HTML is rendered by the browser.
2. `index.html` is the common website entry point.
3. `!` + Enter and `html:5` help generate HTML boilerplate.
4. `head` and `body` have different purposes.
5. Live Server makes HTML practice faster.
6. `h1-h6`, `p`, and `button` are basic content tags.
7. Attributes such as `src`, `controls`, `height`, `width` and `poster` are essential for multimedia.
8. `./asset/...` is a relative-path pattern demonstrated in the project.
9. `audio`, `img`, and `video` let the browser handle much of the media functionality.
10. JavaScript can select HTML using an ID and change its content.
11. Learn the fundamentals before relying heavily on AI.
12. Practice by building the multimedia page from scratch.

Day 03 — Text Content Tags, Lists & Personal Portfolio

TAP Academy — HTML & CSS

Day 03: Text Content Tags, Lists & Personal Portfolio

1. Review: HTML, CSS & JavaScript

- HTML → structure and content.
- CSS → appearance, layout, spacing, colors, animation/presentation.
- JavaScript → behavior and interaction.
- HTML elements use tags; extra information is provided through attributes.
- Examples: src, alt, width, height, controls, autoplay, loop, muted.

2. Images and the alt Attribute

- img displays an image.
- src specifies the image source.
- alt provides alternate text when the image cannot be displayed.
- The alternate text can appear when the image is missing, corrupted, or unavailable.
- alt is important for accessibility and should be provided for images.

3. Video Attributes

- controls → displays playback controls.
- autoplay → starts playback automatically.
- loop → restarts the video after it ends.
- muted → disables audio.
- A common background-video pattern is autoplay + loop + muted, usually without controls.
- Large video files may load or behave differently depending on browser, file size, and other factors.

4. Responsive Video Width

- Avoid unnecessary horizontal scrolling.
- Vertical scrolling is normal; horizontal scrolling should generally be avoided.
- Width can be specified in pixels or percentages.
- width="100%" uses the available width.
- width="50%" uses approximately half the available width.

5. Modern Websites and AI

- Modern portfolio websites can include animations, interactions, immersive scrolling, videos, and advanced visual effects.

- HTML and CSS remain fundamental even when frameworks such as React, Vue, or Next.js are used.
- AI can generate and modify interfaces, but HTML/CSS knowledge is needed to understand the output and communicate requirements clearly.
- Practice recreating designs instead of only copying generated code.
- Explore quality website galleries for design inspiration and inspect how structure, styling, and interactions are implemented.

6. HTML Entities

HTML entities represent special characters and symbols that may not be directly available on a keyboard.

Examples:

- Copyright, registered trademark, trademark
- Currency symbols such as ₹
- Mathematical symbols
- Punctuation and other special symbols
- Emojis

Key points:

- Entity codes commonly begin with & and end with ;.
- Decimal and hexadecimal entity codes can be used.
- Do not memorize every entity code.
- Search for the required HTML entity when you need a special character.

7. Live Preview Extension

- Live Preview by Microsoft provides a browser-like preview inside VS Code.
- Right-click the HTML file → Show Preview.
- It updates while you edit and is useful for quick testing.
- For the full browser experience, test in an actual browser as well.

8. HTML Boilerplate

- Type ! and press Enter in VS Code to generate the standard HTML boilerplate.
- Boilerplate = predefined starting structure.
- DOCTYPE html → declares HTML5.
- html → root element.
- head → metadata/document information.
- body → visible page content.
- title → browser tab/page title.
- Understand the purpose of the boilerplate instead of memorizing it.

9. The lang Attribute

- lang identifies the language of webpage content.
- It is normally placed on the root html element.
- Example: `html lang="en"`.

- Language values can represent English, Telugu, Hindi, Kannada, Chinese, etc.
- Correct language information can help browsers with translation and language-related behavior.
- Unicode allows webpages to display content in many languages.

10. Favicon / Website Icon

- A favicon is the small icon shown near the webpage title/tab.
- Configure it inside head using the link tag.
- Use rel="icon" and href="..."
- The source can be a local image or image URL.

11. Metadata and the head

- The head contains metadata and other document-level information.
- Metadata means data about data.
- Common elements include title, link, and meta.
- The body contains the content users normally see.

12. Browser Developer Tools

Developer Tools are essential for front-end development.

Important areas:

- Elements → inspect and modify HTML.
- Styles → inspect and temporarily modify CSS.
- Console → test JavaScript.
- Network → inspect requests, responses, resources, and status codes.
- Performance → analyze performance and timing.
- Lighthouse → audit website-related quality/performance.
- Other panels include Security and additional diagnostics.

Ways to open Developer Tools:

- Right-click → Inspect.
- Browser menu → More tools → Developer tools.
- F12 in supported browsers.

13. Inspecting HTML, CSS and JavaScript

- Use the inspection tool to identify which HTML element creates visible content.
- Elements shows HTML structure.
- Styles shows related CSS.
- CSS changes made through Developer Tools are temporary.
- Refreshing normally restores the original server-delivered version.
- Developer Tools are useful for debugging and understanding real websites.

14. Important Text Content Tags

- h1 to h6 → headings.
- p → paragraph.
- strong → important/strongly emphasized text.
- em → emphasized text.
- mark → highlighted text.
- del → deleted/struck-through text.
- ins → inserted/underlined text.
- small → smaller text.
- br → line break.
- hr → horizontal thematic break.
- span → inline grouping/styling.
- pre → preformatted text.
- q → short quotation.
- abbr → abbreviation.
- sup → superscript.
- sub → subscript.
- code → code content.
- div → generic block-level container.

Not every tag needs to be memorized immediately. Learn each tag when its purpose appears in a project.

15. strong vs b

- b can visually make text bold.
- strong commonly appears bold but also carries semantic importance.
- Semantic HTML helps accessibility tools such as screen readers understand meaning.
- Use strong when the text is important, rather than using bold only for appearance.

16. Multimedia Tags and Attributes

Image: src, alt, width, height.

Audio: src, controls, autoplay, loop, muted.

Video: src, controls, autoplay, loop, muted, poster, width, height.

Practice these attributes so their behavior becomes familiar.

17. Types of HTML Lists

There are three main list types.

Ordered List

- Use ol.
- Sequence/order matters.
- Items use li.

Unordered List

- Use ul.
- Order does not matter.
- Items use li.

Description List

- Use dl.
- Useful for term/definition-style content.
- dt → term.
- dd → description.

18. Ordered List Attributes

The ol element supports:

- type → numbering style:
 - 1 → numbers
 - A → uppercase letters
 - a → lowercase letters
 - I → uppercase Roman numerals
 - i → lowercase Roman numerals
- start → starting value.
- reversed → descending order.

19. Unordered List Styles

Common marker styles:

- disc
- circle
- square

These are generally controlled through CSS.

To remove bullets, CSS can use:

list-style: none;

20. Nested Lists

A list can contain another list inside an individual li.

Example structure:

- Frontend
 - HTML
 - CSS
 - JavaScript

Nested lists are useful for categories, menus, navigation structures, and hierarchical information.

21. Lists in Real Websites

Lists are commonly used for:

- Navigation bars
- Menus
- Categories
- Product lists
- Documentation
- Sidebars
- Search results

Navigation bars often contain list items with links inside them. CSS can remove bullets and arrange the items horizontally.

22. Emmet Shortcuts in VS Code

Emmet provides shortcuts for quickly generating HTML.

Examples:

- `! →` HTML boilerplate.
- `li*10 →` generates 10 list items.
- Emmet can generate opening/closing tags and nested structures.

Use shortcuts to reduce repetitive typing, but understand the HTML structure being generated.

23. Practice Project: Personal Portfolio / Life Events

Build the portfolio-style project from scratch.

Suggested structure:

1. Main portfolio heading.
2. A section such as Major Life Incidents / Events.
3. Use an ordered list for events that follow a sequence.
4. Use unordered lists for categories or sub-items.
5. Use nested lists where information has levels.
6. Experiment with different ordered-list types.
7. Add meaningful personal/project information instead of blindly copying the instructor's content.

24. Practice Requirements

Continue practicing the earlier multimedia project:

- Add meaningful text.
- Add at least 5 audio elements.
- Add at least 5 images.
- Add at least 5 video elements.
- Practice appropriate attributes such as `src`, `alt`, `controls`, `width`, `height`, `autoplay`, `loop`, and `muted`.
- Build the project yourself instead of copying the complete solution.

25. Important Developer Habit

- Inspect real websites to understand their structure.
- Do not use Developer Tools to cheat on assignments or copy solutions.
- Practice typing and building the structure yourself.
- AI can generate code, but understanding the code is necessary to debug, modify, and explain it.

26. Key Takeaways

1. HTML provides structure and content; CSS controls presentation; JavaScript controls behavior.
2. alt provides alternate text for images and is important for accessibility.
3. Video can use autoplay, loop, muted, and controls.
4. Avoid unnecessary horizontal scrolling.
5. HTML entities provide special symbols without memorizing every code.
6. lang identifies the webpage language and can support browser translation behavior.
7. link rel="icon" is used for favicons.
8. Developer Tools help inspect HTML, CSS, JavaScript, network activity, and performance.
9. Semantic tags such as strong provide meaning in addition to visual formatting.
10. Lists are created with ol, ul, and dl, with li used for list items.
11. Ordered lists support type, start, and reversed.
12. Lists can be nested to represent hierarchical information.
13. Emmet shortcuts make HTML development faster.
14. Build projects from scratch so AI becomes a productivity tool rather than a replacement for understanding.
15. The next class is expected to cover deployment of the projects before continuing to the next project.

TAP Academy — HTML & CSS — Day 04: Hypertext, GitHub Deployment & Portfolio

TAP Academy — HTML & CSS

Day 04: Hypertext, GitHub Deployment & Portfolio

1. Review: HTML Lists

- Ordered list → ol with li items.
- Unordered list → ul with li items.
- Description list → dl with dt (term/title) and dd (description/definition).
- Description lists do not use li.

2. What Is Hypertext?

- HTML stands for HyperText Markup Language.
- Markup language → a language used to structure/place content for display in a browser.
- Hypertext → text/content that can be clicked to navigate or perform an action.
- The a (anchor) tag is used to create hyperlinks.

3. GitHub as Cloud Storage for Code

- Local files on your computer cannot normally be accessed directly by recruiters or other users.
- GitHub provides online/cloud storage for project source code.
- A GitHub repository is similar to a project folder containing the files for a project.
- Repository is commonly shortened to repo.

4. GitHub Repository

1. Create/sign in to a GitHub account.
2. Create a new repository.
3. Give the repository a name.
4. Add an optional description.
5. Use a public repository when you want recruiters/others to view the project.
6. Upload the project files.
7. Commit the changes.

5. Public vs Private Repository

- Public repository → anyone can view the repository.
- Private repository → access is restricted to permitted users.
- Student portfolio projects can be public when the goal is to showcase work.

6. index.html as the Website Entry Point

- For a simple website deployment, the main page should be named exactly index.html.
- Use lowercase index.html.
- Other HTML pages can have different names.
- index.html is commonly used as the default/entry page for a static website.

7. GitHub Pages Deployment

1. Create a repository.
2. Upload index.html.
3. Commit the changes.
4. Open repository Settings.
5. Open Pages.
6. Choose deployment from a branch.
7. Select the main branch.
8. Use the root folder.
9. Save the settings.
10. Wait for deployment to complete.
11. GitHub Pages provides a public website link.

8. GitHub Branches — Basic Idea

- A branch is a separate line of development within a project.
- Different features can be organized into different branches.
- The default branch commonly used for a simple project is main.
- For a basic single-page student project, the main branch is sufficient.

9. Static Deployment File Limits

- The class demonstrated a 25 MB limit for an individual uploaded file through the GitHub web interface.
- Large local media such as trailers/videos may exceed the upload limit.
- If media is not uploaded to the deployed project, it may work locally but fail online.
- Keep required assets inside the deployed project and within the platform's supported limits.

10. Files vs Folders in GitHub Web Upload

- A folder is a collection/container for files.
- The demonstrated GitHub web upload interface accepts files rather than directly uploading a complete folder.
- Project repositories can still contain folders when files are organized through the appropriate Git workflow.

11. Netlify Deployment

- Netlify is another platform for deploying static websites.
- It can be connected using an account such as GitHub.
- A project can be uploaded directly through the web interface.
- Netlify can accept a project folder for deployment.

- The project should contain index.html.
- After upload, Netlify provides a public deployment link.
- The project/site name can be customized, subject to availability.
- A custom domain can also be connected, but it is not required for basic deployment.

12. GitHub vs Netlify

- GitHub is primarily a platform for hosting/managing source-code repositories.
- GitHub Pages can publish static websites from repositories.
- Netlify focuses strongly on easy web deployment and can deploy a complete static project folder.
- Git and GitHub are different concepts; Git commands will be covered later.

13. Why CSS Matters

- HTML provides page structure and content.
- Without CSS, a page appears as basic, unstyled HTML.
- Images may appear at their natural/default dimensions.
- CSS controls layout, sizing, spacing, colors, typography, animation and visual presentation.
- Browser Developer Tools can temporarily remove/inspect CSS to compare styled and unstyled versions.

14. Portfolio Website

- A portfolio can be understood as a website version of your resume.
- A portfolio can include About Me, Skills, Projects, Experience, Contact, Resume and social/profile links.
- HTML creates the structure first.
- CSS can later make the portfolio visually polished.
- Navigation items can link to sections of the same page or to other pages.

15. Creating a Basic Portfolio

Suggested structure:

1. HTML boilerplate.
2. Page title such as My Personal Portfolio.
3. Optional favicon.
4. Main name/heading using h1.
5. Introductory paragraph.
6. Navigation list.
7. Horizontal separator using hr.
8. Main sections using h2.
9. Subsections using h3/additional heading levels.
10. Skills, projects and contact information.
11. Social/profile images or links.
12. Copyright/footer content.
13. Back-to-top link.

16. Heading Hierarchy

- h1 → main page heading.
- h2 → major sections.
- h3 → subsections.
- Additional heading levels can be used for deeper structure.
- Example: h1 → personal name; h2 → About Me, Skills, Projects, Contact; h3 → Technical Skills, Tech Glossary.

17. HTML Text Formatting

- strong → strong semantic importance; commonly displayed bold.
- em → emphasis; commonly displayed italic.
- p → paragraph.
- pre → preformatted text.
- hr → horizontal thematic break.
- br → line break.
- span → inline grouping.
- code → code content.
- abbr → abbreviation.
- Tags can be nested when content needs multiple meanings or formatting.

18. p vs pre

- p displays normal paragraph content.
- Whitespace and line breaks typed inside a normal paragraph are generally collapsed.
- pre preserves whitespace and line breaks as written.
- Use pre when preserving exact text layout is required.
- For normal website paragraphs, use p.

19. Abbreviations with abbr

- abbr represents an abbreviation.
- The title attribute can provide the full form.
- Example: visible text API; title = Application Programming Interface.
- Hovering over the abbreviation can display the full form in supporting browsers.

20. Portfolio Navigation with Anchor Tags

General structure: a href="destination" → clickable text → /a.

- The text/content between opening and closing a tags becomes clickable.
- href specifies the destination/reference.
- href means hypertext reference.
- The destination can be an external website URL, another page/file, a section within the current page, or a file such as a PDF.

21. Making Different Content Clickable

Anything that needs to act as a hyperlink can be placed inside an anchor:

- Text.
- Images.
- Project names.
- Social-media icons.
- Profile links.
- Files/documents.
- Other suitable HTML content.
- For separate clickable images, use separate anchor elements when they have different destinations.

22. External Links

Example pattern: a href=“<https://example.com>” → Visit Website → /a.

- Copy the correct destination URL.
- Place it in href.
- Put the desired clickable text/content between the anchor tags.

23. Opening a Link in a New Tab

- By default, a hyperlink normally opens in the current tab.
- target=“_blank” tells the browser to open the destination in a new browsing context, commonly a new tab.
- Users can also manually open links in a new tab through Ctrl+Click or the context menu.

24. Link States

Default browser styling commonly distinguishes:

- Unvisited link → blue.
- Visited link → purple.
- Active link → red while being activated.
- These colors are browser defaults and can be customized with CSS.

25. Same-Page Navigation with IDs

1. Give the target element a unique id.
 2. Put the ID in the link's href using #.
Example concept: h2 id=“about” → About Me → /h2, then a href=“#about” → About Me → /a.
- The browser jumps to the element with that ID.
 - IDs should be unique within the page.
 - Useful for About Me, Skills, Projects and Contact navigation.

26. ID and URL Concepts

- External websites have URLs that identify their location.
- A section inside your own page can be identified with an id.
- The # fragment in an href points to an element ID on the current page.

27. Downloading and Opening Files

Example: a href=“./resume.pdf” → View Resume → /a.

- A browser may open supported documents such as PDFs directly.
- target=“_blank” can open the document in a new tab.
- The download attribute can request that the linked file be downloaded.

28. Custom Download Filename

- The download attribute can include a filename value.
- Example concept: download=“my-resume.pdf”.
- Actual download behavior can depend on browser and server behavior.

29. Email Links

Use the mailto scheme: a href=“mailto:example@email.com” → example@email.com → /a.

- Clicking the link opens the user’s configured mail application/service.

30. Phone Links

Use the tel scheme: a href=“tel:+911234567890” → +91 1234567890 → /a.

- On a device with calling support, clicking can open the phone/dialer application.
- Desktop behavior depends on installed applications and system configuration.

31. Portfolio Social Links

- A portfolio can include LinkedIn, GitHub, Instagram or other social platforms.
- Social icons/images can be placed inside anchor tags so clicking the image opens the corresponding profile.

32. HTML Resources for Icons

- The class demonstrated FlatIcon as a source for icons and brand logos.
- When using external assets, use appropriate image URLs or download assets into the project according to the site’s usage/licensing terms.
- HTML’s img tag can display the selected icon.

33. VS Code Prettier

- Prettier is a code-formatting extension commonly used in VS Code.
- It formats HTML/code consistently and improves indentation/readability.
Useful settings:
 1. Install the Prettier extension.
 2. Enable Format On Save.
 3. Set Prettier as the Default Formatter.
- Format On Type was mentioned as another option, but Format On Save was preferred in the class.

34. Emmet and Multicursor

- Emmet can generate repetitive HTML structures quickly.
- Example: `li*5` generates five list items.
- VS Code multicursor can be used with `Alt+Click` to edit multiple locations at once.

35. Portfolio Practice Requirements

Build the portfolio from scratch and include:

- Personal name and introduction.
- Navigation links.
- About Me section.
- Technical Skills.
- Tech Glossary using abbr.
- Projects.
- Contact information.
- Resume link.
- LinkedIn/GitHub/social links.
- Clickable images/icons.
- Same-page navigation using IDs.
- Email and phone links.
- Footer/copyright.
- Back-to-top link.

36. Deployment Practice

1. Make sure `index.html` exists.
2. Keep required local assets inside the project.
3. Deploy using GitHub Pages or Netlify.
4. Open the deployed link on a desktop browser.
5. Test the deployed link on a mobile phone.
6. Verify images, audio/video, links, navigation, resume and contact actions work online.
7. Remember that local files that were not uploaded will not work after deployment.

37. AI and Fundamentals

- AI can generate or deploy code quickly.
- Developers still need to understand the process so they can troubleshoot when generated code or automated deployment fails.
- Use AI as a productivity tool after understanding what the generated code is doing.

38. Key Takeaways

1. Hypertext means clickable/connected content that can navigate to another resource or section.
2. The `a` tag creates hyperlinks.
3. `href` specifies the hyperlink destination.

4. target="_blank" opens a link in a new tab/browsing context.
5. GitHub repositories provide online storage and collaboration for project code.
6. GitHub Pages can publish a static website from a repository.
7. index.html is the common entry page for a static website.
8. Netlify provides another simple way to deploy static websites.
9. HTML creates structure; CSS controls presentation.
10. id + #id enables same-page section navigation.
11. download can request file downloading and can provide a suggested filename.
12. mailto: creates email links.
13. tel: creates phone links.
14. abbr can provide the full form of an abbreviation through title.
15. p and pre behave differently with whitespace and line breaks.
16. Prettier can automatically format HTML when configured as the default formatter and used with Format On Save.
17. Build and deploy the portfolio yourself, then test it on both desktop and mobile.

1. HTML/CSS_D05 Tables [25-09-2026] - YouTube

Verified Tools & Resources

Tool	Link	Use
Superhuman Docs	superhuman.com/docs	Free tool to store notes (preserves code formatting better than Word/Google Docs)
Awwwards	awwwards.com	Top website designs and creative layout inspiration
DESIGN.md	designmd.ai	Design language references and directory
Refero Design	refero.design	Real-world UI layouts and design pattern references
Magic UI	magicui.design	Ready-made UI components (copy-paste code for docks, animated text, tweet cards)
Aceternity UI	ui.aceternity.com	Modern React components (cards, effects, notches)
React Bits	reactbits.dev	More component ideas and interactive UI animations
Color Hunt	colorhunt.co	Color palette (palette = color combo) ideas
Color Space	mycolor.space	Generate color themes & CSS gradients
Coolors	coolors.co	Quick palette generation and contrast check

Key Concepts (Simple Explanations)

- **Hypertext:** Clickable content (text, image, or button) used to redirect users to a website, another page, an email app (`mailto:`),

phone dialer (`tel:`), or a bookmark on the same page.

- **Table Basics:** A table organizes data into rows and columns:
 - `<table>` = Main container.
 - `<tr>` = Table Row (horizontal line).
 - `<th>` = Table Header (bold, centered text).
 - `<td>` = Table Data (normal cell text).
- **Deprecated Attributes:** Attributes like `border`, `cellpadding` (inner space), and `cellspacing` (gap between borders) work in HTML but are deprecated; use CSS instead.
- **Merging Cells:**
 - `colspan` = Combines cells sideways across columns.
 - `rowspan` = Combines cells up-down across rows.
- **Semantic Structure:** Splitting tables into `<thead>`, `<tbody>`, `<tfoot>`, and adding a `<caption>` title makes the structure cleaner, accessible, and SEO-friendly.
- **Iframe (`<iframe>`):** Used to embed external content inside a webpage (Google Maps, YouTube videos, other websites). You must use official embed links rather than standard browser URLs.

Important Code (code = instructions for computer)

1. Basic Table Structure

```
<table>
  <tr>
    <th>Name</th>
    <th>Language</th>
    <th>Year</th>
  </tr>
  <tr>
    <td>James</td>
    <td>Java</td>
    <td>1991</td>
  </tr>
  <tr>
    <td>Guido</td>
    <td>Python</td>
    <td>1991</td>
  </tr>
</table>
```

2. Column Span (colspan = merge cells sideways)

```
<table border="1">
  <tr>
    <th>Name</th>
    <!-- Merges 3 columns sideways for skills -->
    <th colspan="3">Skills</th>
  </tr>
```

```

<tr>
  <td>James</td>
  <td>C</td>
  <td>Java</td>
  <td>HTML</td>
</tr>
</table>

```

3. Row Span (rowspan = merge cells up-down)

```

<table border="1">
  <tr>
    <th>Name</th>
    <th>Companies</th>
  </tr>
  <tr>
    <!-- Merges 3 rows down for James -->
    <td rowspan="3">James</td>
    <td>Sun Microsystems</td>
  </tr>
  <tr>
    <td>Oracle</td>
  </tr>
  <tr>
    <td>Liquid Robotics</td>
  </tr>
</table>

```

4. Table Head/Body/Foot + Caption

```

<table border="1">
  <caption>Famous People</caption>
  <thead>
    <tr>
      <th>Name</th>
      <th>Field</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>James</td>
      <td>Java</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td colspan="2" style="text-align: center;">Note: Tech Pioneers</td>
    </tr>
  </tfoot>
</table>

```

```
</tfoot>
</table>
```

5. Multi-Page Linking & Smooth Scroll (CSS)

```
<!-- Link to another page -->
<a href="greet.html">Click here for greetings</a>

<!-- Smooth scrolling for entire webpage -->
<style>
  html {
    scroll-behavior: smooth;
  }
</style>

<!-- Jump back to top -->
<a href="#">Back to Top</a>
```

6. Embed Google Map

```
<!-- Use Share -> Embed Map option from Google Maps -->
<iframe
  src="[https://www.google.com/maps/embed?
pb=!1m18!1m12!1m3!1d3888.898863683244!2d77.6083162758826!3d12.914207916127113!2m3!1f0!2f0!3f0!3
m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x3bae1501b876d755%3A0xb304b50c4aa3e4ee!2sTAP%20ACADEMY%20BTM!
5e0!3m2!1sen!2sin!4v1727255000000](https://www.google.com/maps/embed?
pb=!1m18!1m12!1m3!1d3888.898863683244!2d77.6083162758826!3d12.914207916127113!2m3!1f0!2f0!3f0!3
m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x3bae1501b876d755%3A0xb304b50c4aa3e4ee!2sTAP%20ACADEMY%20BTM!
5e0!3m2!1sen!2sin!4v1727255000000)"
  width="900"
  height="350"
  style="border:0;"
  allowfullscreen=""
  loading="lazy">
</iframe>
```

7. Embed YouTube Video

```
<!-- Use Share -> Embed option from YouTube (direct links will fail) -->
<iframe
  width="560"
  height="315"
  src="[https://www.youtube.com/embed/dQw4w9WgXcQ](https://www.youtube.com/embed/dQw4w9WgXcQ)"
  title="YouTube video player"
  frameborder="0"
  allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-
picture; web-share"
```

```
allowfullscreen>  
</iframe>
```

🎯 Placed Students' Advice (Real Interview Insights)

- **Test Automation Engineer at Cisco (4.58 LPA):**
 - Interviewers specifically drilled into: *"How do you connect the frontend to the backend?"* and asked to write the connection code.
 - Attending regular **mock interviews** was the key turning point in removing anxiety and answering confidently.
- **Software Developer (4.2 LPA):**
 - Cleared a 250-question contest followed by a strict 20-minute live coding round (writing, running, and verbally explaining the logic line by line).
 - Major focus was on **Core OOPs concepts** and **SQL queries (specifically JOINS)**.
 - **Warning on AI use in interviews:** Avoid using AI to copy code in proctored coding assessments; anti-cheating systems detect it immediately, and live interviewers test whether you can explain every single line.
 - Consistent hands-on coding practice of **6 hours every day** is what delivers results.